

Day 42 — Line-by-Line Syntax Explanation

day42_deep_cnn_stacking.py, segment by segment, line by line. Pure Python/NumPy syntax focus, with the math each chunk implements, a system diagram of how it works, and real (rerun, not fabricated) graphs showing the code+math turned into a visualization.

MD Mutasim Billah | 52-week Data Science → ML/AI roadmap | Week 7, Day 1 reference

This document walks the script top to bottom in the order it actually runs. Each segment matches one numbered section of the script. Wherever a code chunk implements a real formula (multi-channel convolution's sum, the receptive-field recursion, or a network's parameter count), a purple **MATH** box shows that formula immediately before the code, followed by the syntax explanation. After each major segment, a green **SYSTEM DIAGRAM** shows how that math and code fit together as a working system — including the two-block training work cycle, and the script's own real output graphs.

MATH box legend: $\times$ = matrix multiply / elementwise times W, b = filter weights/bias m = batch size k = kernel size C_{in}/C_{out} = input/output channel count r, j = receptive field, jump

Segment 0 — Imports and Global Setup

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

np.random.seed(42)
```

Identical setup pattern to Days 36-41: **np** for all array math, **matplotlib.use("Agg")** before importing **pyplot** so charts render straight to PNG files instead of a GUI window. Nothing new here — today's changes are entirely in what gets built with these same tools.

Segment 1 — Shared Helpers: reused verbatim from Days 39-41

```
def relu(z):
    return np.maximum(0, z)

def drelu(z):
    return (z > 0).astype(z.dtype)

def tanh(z):
    return np.tanh(z)

def dtanh(a):
    return 1 - a ** 2

def softmax(z):
    z_shifted = z - z.max(axis=1, keepdims=True)
    exp_z = np.exp(z_shifted)
    return exp_z / exp_z.sum(axis=1, keepdims=True)
```

relu/drelu/tanh/dtanh/softmax are character-for-character identical to Day 41's versions — today's lesson changes what happens BEFORE and BETWEEN convolution layers (multi-channel sums, a second block), not the pointwise activation math itself, so these five functions needed zero changes.

```
def one_hot(y, num_classes):
    m = y.shape[0]
    encoded = np.zeros((m, num_classes))
    encoded[np.arange(m), y] = 1.0
    return encoded

def cross_entropy_loss(probs, y_onehot, eps=1e-9):
    probs = np.clip(probs, eps, 1 - eps)
    return -np.mean(np.sum(y_onehot * np.log(probs), axis=1))

def he_scale(n_in):
    return np.sqrt(2.0 / n_in)
```

Also unchanged from Day 41: **one_hot** and **cross_entropy_loss** for the 4-class junction task's labels and loss, and **he_scale** reused for BOTH OneBlockConvNet's and TwoBlockConvNet's ReLU-activated conv filters and tanh-activated dense layer.

Segment 2 — conv2d_forward_mc and conv2d_backward_mc

MATH

$$Z[:,f,i,j] = \text{sum over } c,a,b \text{ of } X[:,c,i+a,j+b] \times W[f,c,a,b] + b[f]$$

```
def conv2d_forward_mc(X, W, b):
    m, C_in, H, Win = X.shape
    C_out, C_in_w, kH, kW = W.shape
    out_H = H - kH + 1
    out_W = Win - kW + 1
    Z = np.zeros((m, C_out, out_H, out_W))
    for f in range(C_out):
        for i in range(out_H):
            for j in range(out_W):
                patch = X[:, :, i:i+kH, j:j+kW]
                Z[:, f, i, j] = np.sum(
                    patch * W[f], axis=(1, 2, 3)) + b[f]
    return Z
```

The signature difference from Day 41's **conv2d_forward**: **X** now has FOUR dimensions (**m, C_in, H, Win**) instead of three, and **patch = X[:, :, i:i+kH, j:j+kW]** slices ALL input channels at once — **axis=(1, 2, 3)** then sums over channel AND both spatial kernel dimensions together, collapsing all of them into the single output value **Z[:, f, i, j]**.

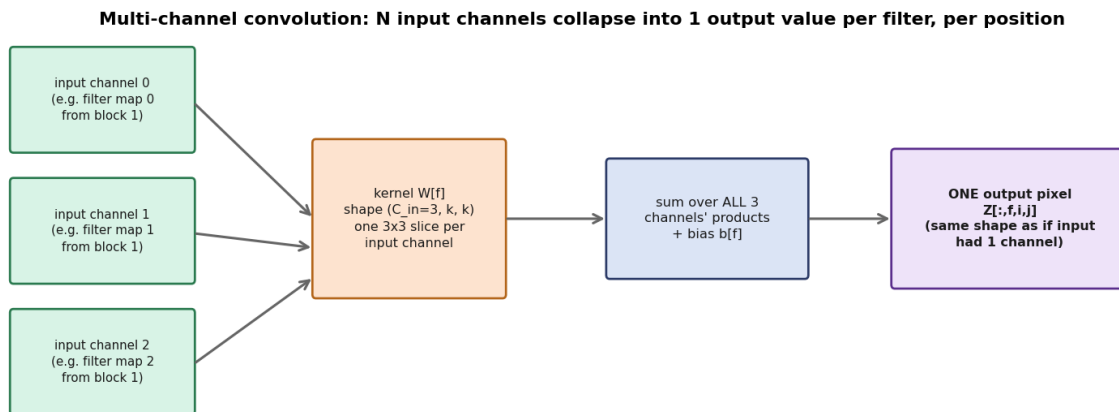
MATH

$$dW[f] = \text{sum}(dZ[:,f,i,j] \times \text{patch, per image}) / m \quad db[f] = \text{sum}(dZ[:,f,i,j]) / m$$
$$dX[:, :, \text{patch region}] += dZ[:,f,i,j] \times W[f] \text{ (now spans ALL } C_{\text{in}} \text{ channels, not just 1)}$$

```
def conv2d_backward_mc(dZ, X, W):
    ...
    dW = np.zeros_like(W)
    db = np.zeros(C_out)
    dX = np.zeros_like(X)
    for f in range(C_out):
        for i in range(out_H):
            for j in range(out_W):
                patch = X[:, :, i:i+kH, j:j+kW]
                dZ_ij = dZ[:, f, i, j]
                dW[f] += np.sum(
                    dZ_ij[:,None,None,None]*patch, axis=0)/m
                db[f] += np.sum(dZ_ij) / m
                dX[:, :, i:i+kH, j:j+kW] += (
                    dZ_ij[:,None,None,None] * W[f])
```

dZ_ij[:,None,None,None] reshapes a (m,) array into (m,1,1,1) — one more **None** than Day 41's version, matching **patch**'s new 4D shape (m,C_in,kH,kW). Otherwise this is EXACTLY Day 41's accumulation pattern: **dX[...] +=** (not **=**) because output positions still overlap across a multi-channel patch the same way they did for a single channel.

SYSTEM DIAGRAM



Multi-channel convolution: however many input channels there are, they all collapse into ONE output value per filter, per position.

Segment 3 — maxpool_forward and maxpool_backward: reused from Day 41

```
def maxpool_forward(A, size=2, stride=2):
    m, C, H, W = A.shape
    ...
    flat = window.reshape(m, C, size * size)
    idx = flat.argmax(axis=2)
    out[:, :, i, j] = flat[
        np.arange(m)[: ,None], np.arange(C)[None, :], idx]
    ...

def maxpool_backward(dOut, A, masks, size=2, stride=2):
    ...
    dA[:, :, i*stride:i*stride+size,
        j*stride:j*stride+size] += (
        masks[(i, j)] * dOut[:, :, i, j][:, :, None, None]
    )
```

Both functions are copied over UNCHANGED from Day 41 — max pooling already operated on a generic (m, C, H, W) array with an arbitrary channel count **C**, so nothing about stacking a second block requires any modification here. This is a concrete example of Topic 0's 'block' idea: pooling was already general enough to reuse as-is.

Segment 4 — demo_multichannel_conv (gradient check)

```
rng = np.random.RandomState(0)
m = 2
X = rng.randn(m, 3, 7, 7)
W = rng.randn(4, 3, 3, 3) * 0.1
b = rng.randn(4) * 0.1
Z = conv2d_forward_mc(X, W, b)
```

A tiny toy example — 2 images, 3 input channels, 7x7 — makes the shapes concrete before trusting the backward pass: **Z.shape** comes out **(2, 4, 5, 5)**, matching **out_H = out_W = 7-3+1 = 5** for a 4-filter layer.

```
def loss_mean(W_, b_, X_):
    return np.sum(dZ * conv2d_forward_mc(X_, W_, b_)) / m

i, j, k, l = 1, 2, 1, 1
W2 = W.copy(); W2[i, j, k, l] += eps
numgrad_W = (loss_mean(W2, b, X) - loss_mean(W, b, X)) / eps
```

Nudging a SINGLE weight **W2[i, j, k, l] += eps** by a tiny amount and rerunning the forward pass is the standard numerical-gradient recipe used throughout this roadmap — **loss_mean** divides by **m** to match how **dW** is scaled inside **conv2d_backward_mc**, so the numerical and analytical values can be compared directly.

```
def loss_sum(W_, b_, X_):
    return np.sum(dZ * conv2d_forward_mc(X_, W_, b_))

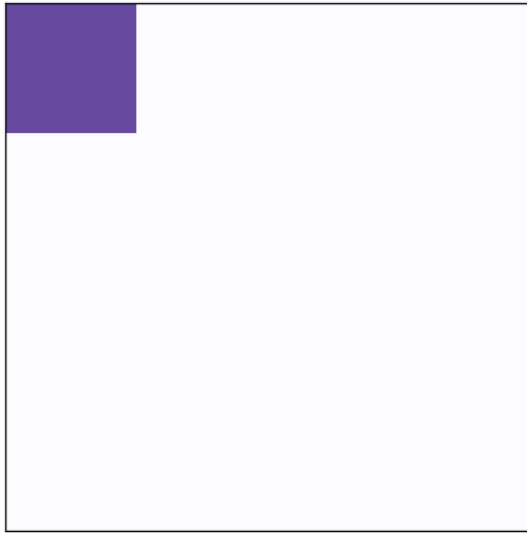
X2 = X.copy(); X2[a, c, d, e] += eps
numgrad_X = (loss_sum(W, b, X2) - loss_sum(W, b, X)) / eps
```

dX, unlike **dW/db**, is NOT divided by **m** inside the backward pass — so its numerical check deliberately uses **loss_sum** (no **/m**) instead of **loss_mean**, matching that different scaling convention exactly, the same subtlety Day 41's single-channel gradient check also had to account for.

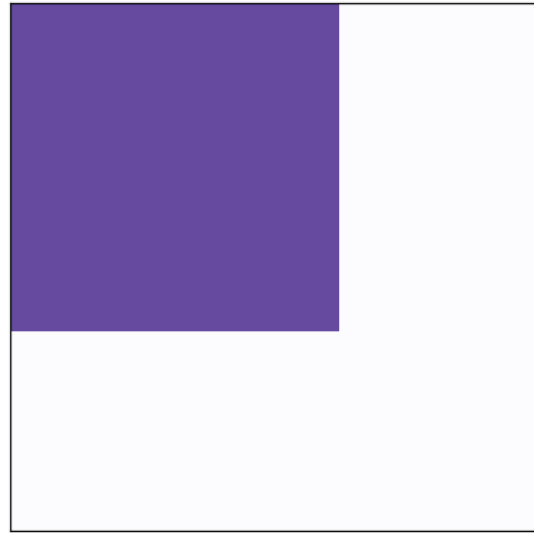
REAL GRAPH (script's own output)

Receptive field of the top-left output unit, on a 16x16 image

1 block: $4 \times 4 = 16$ px



2 blocks: $10 \times 10 = 100$ px



REAL GRAPH (script's own output) -- not from this segment directly, but the same
conv2d_forward_mc/conv2d_backward_mc functions verified here are what the receptive-field sensitivity sweep in Segment
5 relies on being correct.

Segment 5 — receptive_field_growth and the numeric verification sweep

MATH

$$r_{out} = r_{in} + (k-1) \times j_{in} \quad j_{out} = j_{in} \times stride$$

```
def receptive_field_growth(layers):
    r, j = 1, 1
    trace = [(r, j)]
    for k, s in layers:
        r = r + (k - 1) * j
        j = j * s
        trace.append((r, j))
    return r, j, trace

one_block = [(3, 1), (2, 2)]
two_block = [(3, 1), (2, 2), (3, 1), (2, 2)]
```

layers is a list of (**kernel_size**, **stride**) tuples, unpacked in the **for k, s in layers** loop header — the same tuple-unpacking pattern Day 41 used for testing several padding/stride configs. Each iteration updates BOTH **r** (receptive field) and **j** (jump) using the previous iteration's values, making this a genuine recursive accumulation, not an independent per-layer calculation.

```
def avgpool_forward(A, size=2, stride=2):
    ...
    out[:, :, i, j] = window.mean(axis=(2, 3))
    return out
```

A SEPARATE pooling function, used only for this diagnostic — average pooling's **window.mean(axis=(2, 3))** spreads sensitivity across every pixel in the window equally, unlike max pooling's argmax selection, so a numeric sensitivity sweep built on it reveals the FULL theoretical receptive field rather than just one arbitrary winning path.

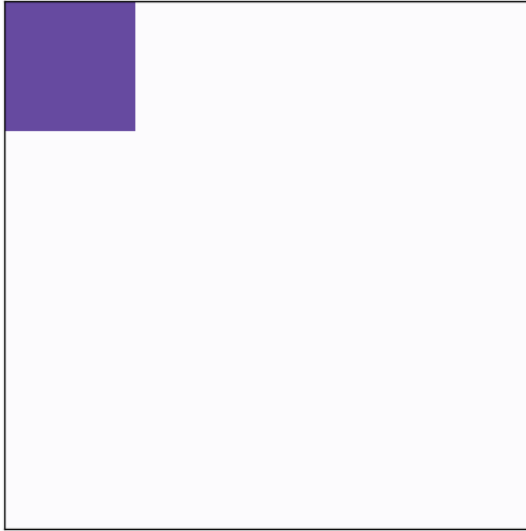
```
for r in range(size):
    for c in range(size):
        Xp = base_X.copy(); Xp[0, 0, r, c] += eps
        if abs(forward_2block(Xp) - base2) > 1e-9:
            sensitive2.append((r, c))
```

A brute-force double loop perturbs EVERY one of the 256 input pixels one at a time and checks whether the top-left output unit's value changed at all — **Xp = base_X.copy()** ensures each test starts from a clean, unperturbed copy so the 256 checks don't interfere with each other.

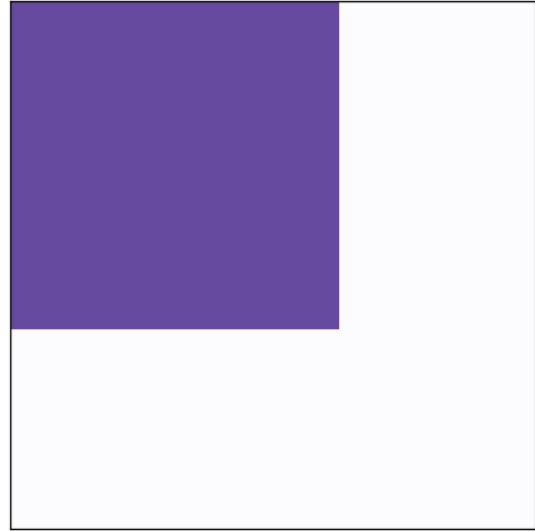
REAL GRAPH (script's own output)

Receptive field of the top-left output unit, on a 16x16 image

1 block: $4 \times 4 = 16$ px



2 blocks: $10 \times 10 = 100$ px



REAL GRAPH (script's own output) -- the top-left output unit's receptive field, highlighted on the original 16x16 image, for one block versus two blocks: exactly the sensitivity sweep coded above.

Segment 6 — make_junction_dataset and demo_dataset

```
dirs_all = [(-1, 0), (1, 0), (0, -1), (0, 1)]
for cls in range(4):
    n_arms = cls + 1
    for _ in range(n_per_class):
        im = rng.randn(size, size) * noise
        r = rng.randint(margin, size - margin)
        c = rng.randint(margin, size - margin)
        if n_arms == 4:
            chosen = dirs_all
        else:
            idx = rng.choice(4, size=n_arms, replace=False)
            chosen = [dirs_all[i] for i in idx]
```

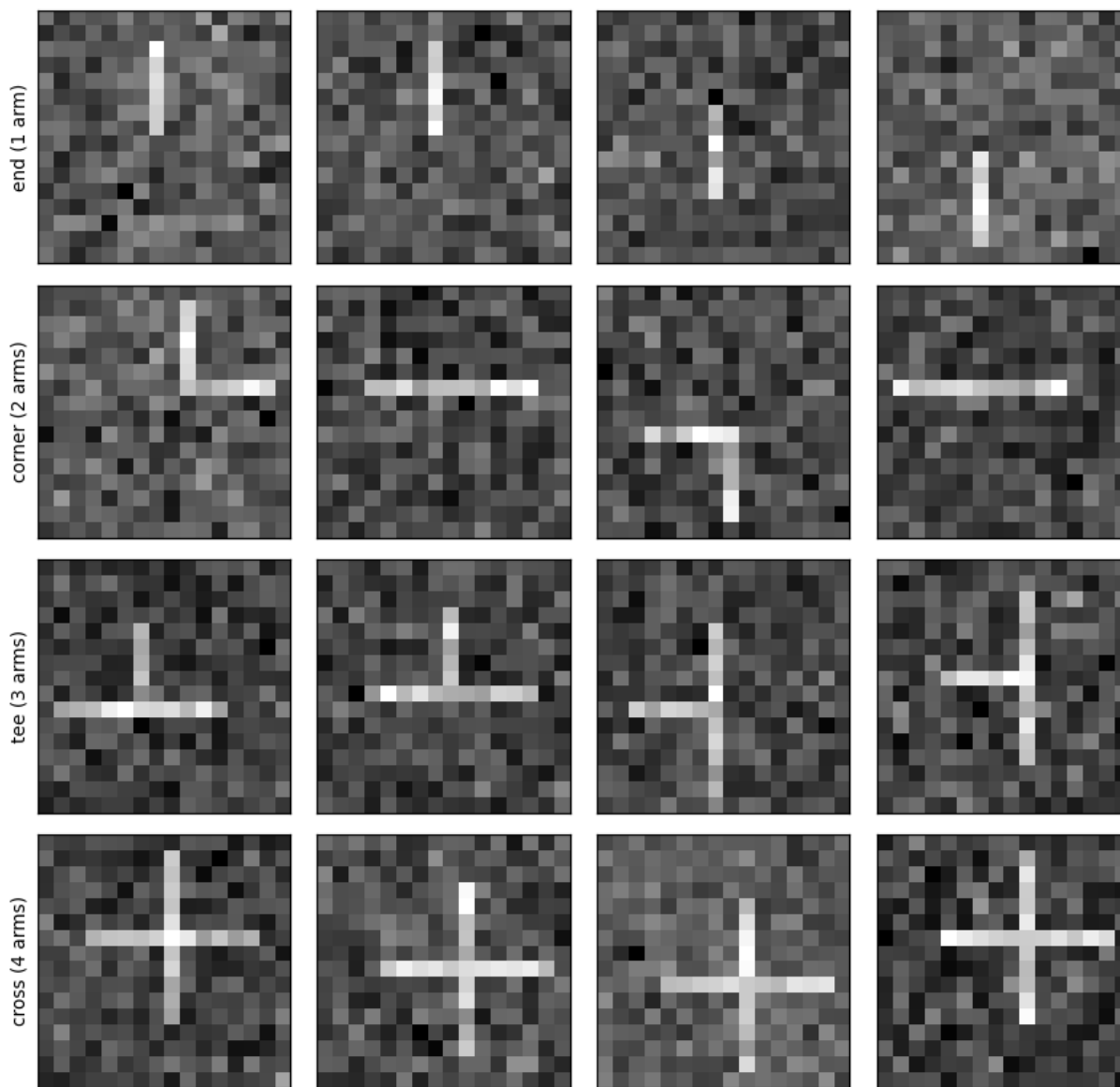
cls + 1 maps class index 0-3 directly to arm count 1-4 — a compact way to tie the loop variable to the dataset's meaning. **rng.choice(4, size=n_arms, replace=False)** picks **n_arms** DIFFERENT directions out of the 4 available without repeats, so a 'corner' (2 arms) gets 2 distinct directions, never the same direction twice.

```
for (dr, dc) in chosen:
    for step in range(1, arm + 1):
        rr, cc = r + dr * step, c + dc * step
        im[rr, cc] += 2.0
im[r, c] += 2.0
```

The nested loop walks each CHOSEN direction outward from the center point: **r + dr*step, c + dc*step** is the same direction-times-distance offset pattern used for drawing lines in many small graphics routines. **im[r, c] += 2.0** brightens the center point itself once, after all arms are drawn, so every image has a clearly visible junction point regardless of arm count.

REAL GRAPH (script's own output)

Junction dataset: how many arms meet at the point?



Not a mockup -- this is junction_dataset.png, this exact script rerun fresh: 4 real example images from each class.

Segment 7 — OneBlockConvNet: init, forward, backward

```
class OneBlockConvNet:
    def __init__(self, n_filters=3, k=3, img_size=16,
                  hidden=8, n_classes=4, lr=0.05, seed=42):
        rng = np.random.RandomState(seed)
        self.Wc = rng.randn(n_filters, 1, k, k) * he_scale(k*k)
        self.bc = np.zeros(n_filters)
```

self.Wc is shaped **(n_filters, 1, k, k)** -- the middle **1** is **C_in=1** for a grayscale image, making this the SAME multi-channel-capable **conv2d_forward_mc** function called with the smallest possible channel count, rather than a separate single-channel-only code path.

```
def n_params(self):
    return (self.Wc.size + self.bc.size + self.W1.size
            + self.b1.size + self.W2.size + self.b2.size)
```

.size on a NumPy array returns its TOTAL element count (product of all dimensions) -- summing **.size** across every learnable tensor gives the network's total trainable parameter count directly, without needing to hand-derive the formula separately (though Topic 4's math box does exactly that, as a cross-check).

```
def forward(self, X):
    m = X.shape[0]
    self.X = X[:, None, :, :]
    self.Zc = conv2d_forward_mc(self.X, self.Wc, self.bc)
    self.Ac = relu(self.Zc)
    self.pooled, self.masks = maxpool_forward(self.Ac, 2, 2)
    self.flat = self.pooled.reshape(m, -1)
```

X[:, None, :, :] inserts a new length-1 axis at position 1 -- turning a plain (m, H, W) grayscale batch into the (m, 1, H, W) shape **conv2d_forward_mc** requires. This one-line reshape is what lets the SAME multi-channel conv function serve as this network's very first layer.

Segment 8 — TwoBlockConvNet: init, forward, backward

```
class TwoBlockConvNet:
    def __init__(self, f1=3, f2=6, k=3, img_size=16,
                  hidden=8, n_classes=4, lr=0.08, seed=42):
        ...
        self.Wc1 = rng.randn(f1, 1, k, k) * he_scale(k*k)
        ...
        self.Wc2 = rng.randn(f2, f1, k, k) * he_scale(f1*k*k)
        ...
        self.shapes = dict(out1=out1, pool1=pool1, out2=out2,
                            pool2=pool2, flat_dim=flat_dim)
```

self.Wc1's C_in=1 matches a grayscale image; **self.Wc2's C_in=f1** matches block 1's OUTPUT channel count instead -- this one difference in the second filter tensor's shape is the entire structural change that makes this a two-block network. **self.shapes** stores every intermediate size for printing and later cross-checking against the math box's predictions.

```
def forward(self, X):
    ...
    self.Zc1 = conv2d_forward_mc(self.X, self.Wc1, self.bc1)
    self.Ac1 = relu(self.Zc1)
    self.pooled1, self.masks1 = maxpool_forward(self.Ac1, 2, 2)
    self.Zc2 = conv2d_forward_mc(
        self.pooled1, self.Wc2, self.bc2)
    self.Ac2 = relu(self.Zc2)
    self.pooled2, self.masks2 = maxpool_forward(self.Ac2, 2, 2)
    self.flat = self.pooled2.reshape(m, -1)
```

self.pooled1 -- block 1's OUTPUT -- is fed directly as the **X** argument to the second **conv2d_forward_mc** call, exactly like block 1's own input, just already a stack of 3 feature maps instead of 1 grayscale channel. This single line is what 'stacking' means in code: reusing the same function, one layer's output becoming the next layer's input.

```
def backward(self, y_onehot):
    ...
    dWc2, dbc2, dpooled1 = conv2d_backward_mc(
        dZc2, self.pooled1, self.Wc2)
    dAc1 = maxpool_backward(
        dpooled1, self.Ac1, self.masks1, 2, 2)
    dZc1 = dAc1 * drelu(self.Zc1)
    dWc1, dbc1, _ = conv2d_backward_mc(dZc1, self.X, self.Wc1)
```

dpooled1 -- the THIRD return value of **conv2d_backward_mc** that Day 41's single-block network always discarded as **_** -- is captured here and fed straight into **maxpool_backward** as block 1's incoming gradient. The final call's **_** discards conv1's own dX, since conv1 IS the first layer -- there's nothing earlier to hand it to.

Segment 9 — demo_training: the real one-block vs two-block comparison

```
net1 = OneBlockConvNet(n_filters=3, k=3, img_size=16, hidden=8,
                       n_classes=4, lr=0.05, seed=42)
p1 = net1.n_params()
u1 = net1.accuracy(X, y)
losses1 = net1.train(X, y, epochs=500)
train_acc1 = net1.accuracy(X, y)
test_acc1 = net1.accuracy(X_test, y_test)
```

u1 is captured BEFORE **net1.train(...)** runs -- the same before/after pattern used since Day 39 to prove a reported accuracy reflects genuine learning, not just a lucky random initialization. **net1.n_params()** is called once and stored in **p1**, reused later for both the printed comparison and the bar chart.

MATH

$$\text{reduction \%} = 100 \times (1 - p_{\text{two}} / p_{\text{one}})$$

```
reduction = 100 * (1 - p2 / p1)
print(f"...TwoBlockConvNet does it with {p2} parameters versus "
      f"OneBlockConvNet's {p1}: a {reduction:.1f}% reduction. ...")
```

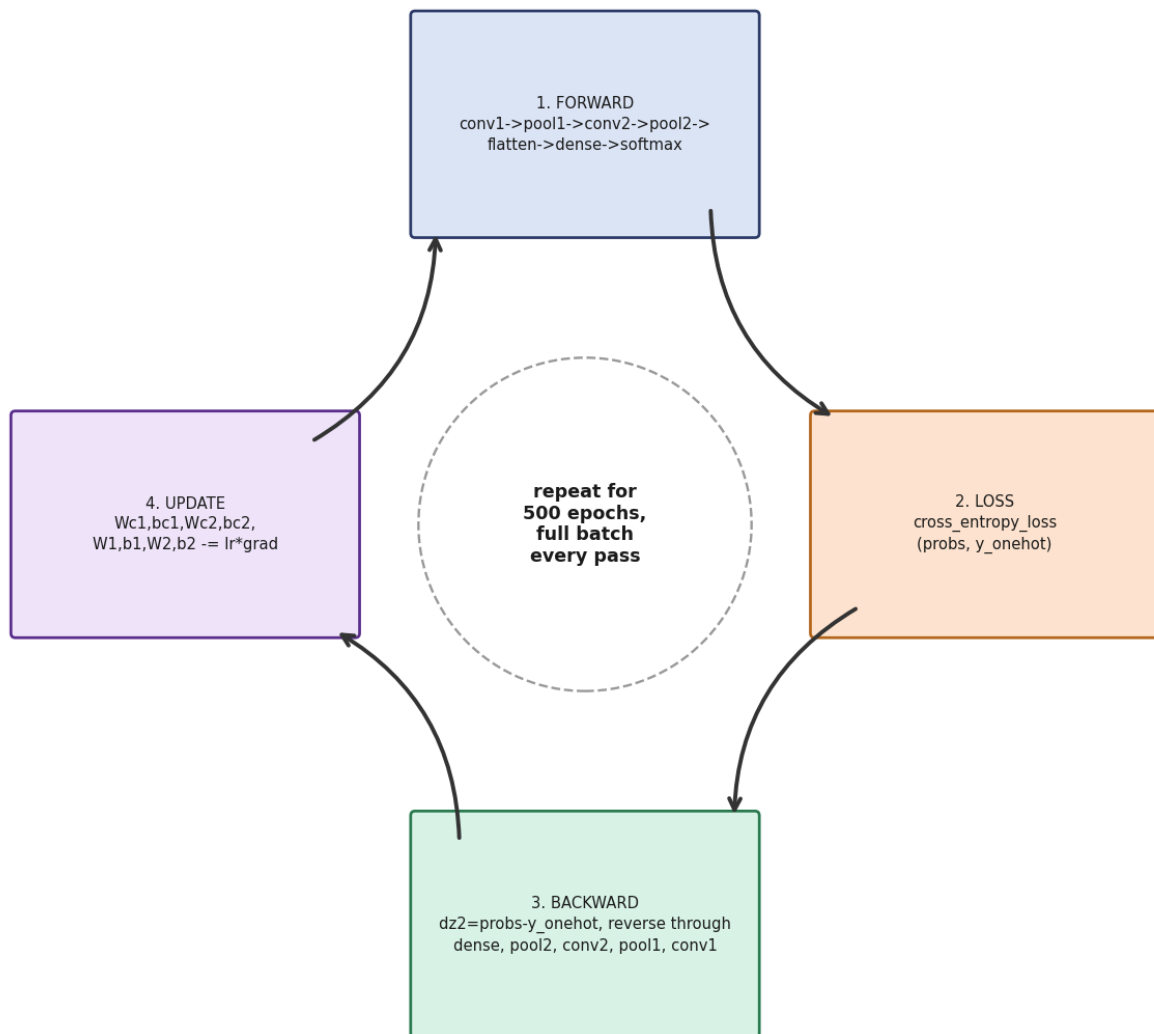
100 * (1 - p2 / p1) is the standard percentage-reduction formula -- computed directly from the two REAL parameter counts gathered above, not hardcoded, so this number would automatically update if the filter counts or hidden sizes above were ever changed.

```
fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(losses1, label=f"OneBlockConvNet ({p1} params)",
        color="#c0392b")
ax.plot(losses2, label=f"TwoBlockConvNet ({p2} params)",
        color="#1a5f3f")
```

Both loss curves are plotted on the SAME axes for direct visual comparison -- the f-string labels embed each network's real parameter count directly into the legend, so the parameter-efficiency story is visible on the graph itself, not just in the printed text.

SYSTEM DIAGRAM

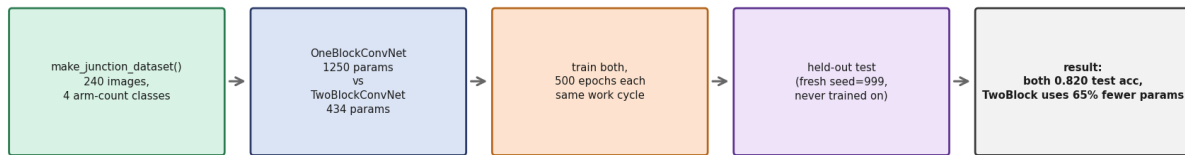
The training work cycle, extended for TWO stacked conv+pool blocks



The training work cycle, extended for TWO stacked conv+pool blocks: forward through both blocks and the dense head, loss, backward through the same chain in reverse, then update all 8 parameter tensors.

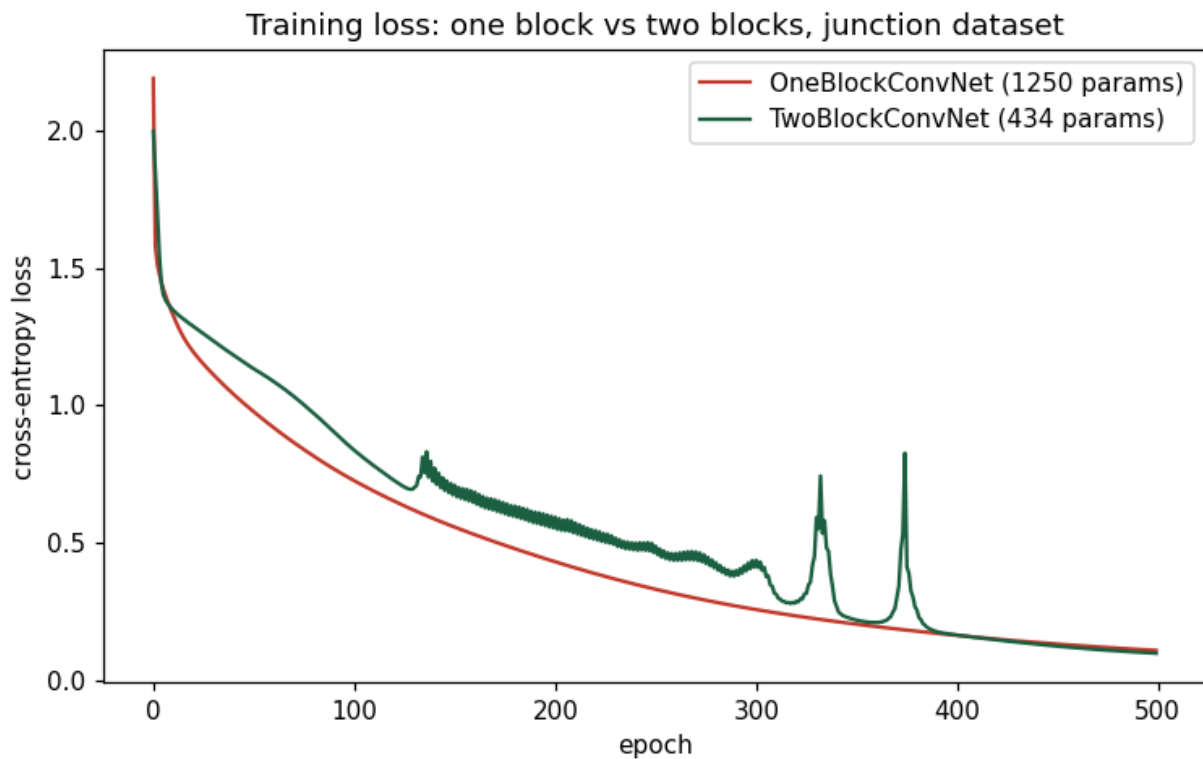
SYSTEM DIAGRAM

Segment 6 pipeline: dataset to a genuine one-block vs two-block comparison



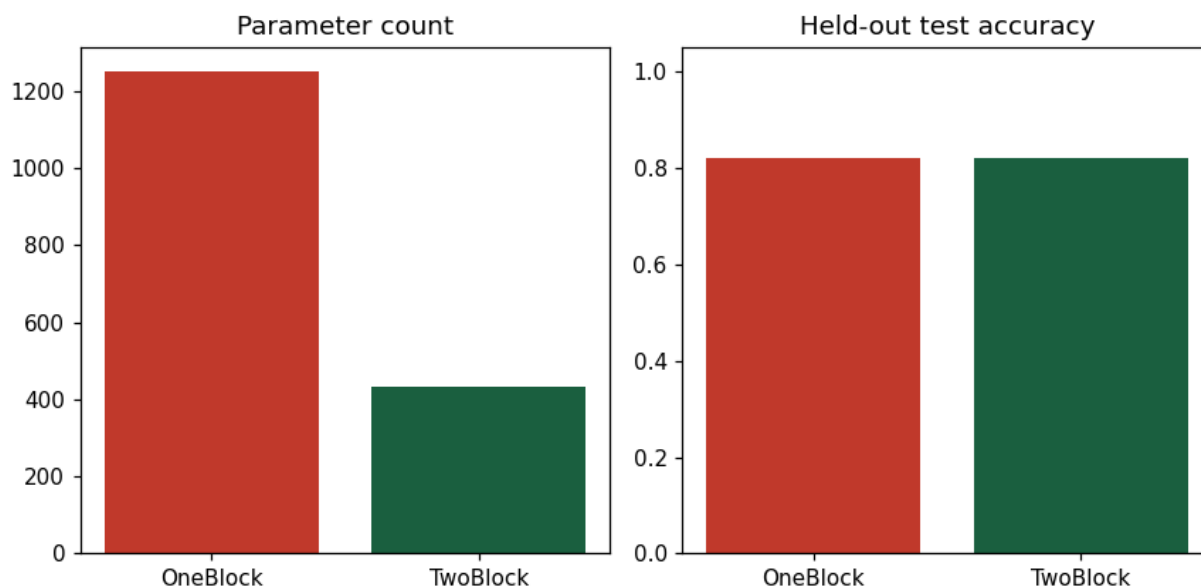
Zooming out: the full comparison pipeline this segment runs end to end, from the junction dataset through training to the honest parameter-efficiency result.

REAL GRAPH (script's own output)



Not a mockup -- this is stacking_comparison.png, this exact script rerun fresh: real training loss for both networks, including TwoBlockConvNet's genuine training instability spikes.

REAL GRAPH (script's own output)



Not a mockup -- this is param_efficiency.png, this exact script rerun fresh: parameter count and held-out test accuracy, side by side, for both networks.

Segment 10 — note_on_pytorch, main, and the Entry-Point Guard

```
model = nn.Sequential(
    nn.Conv2d(in_channels=1, out_channels=3, kernel_size=3),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, stride=2),
    nn.Conv2d(in_channels=3, out_channels=6, kernel_size=3),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, stride=2),
    nn.Flatten(),
    nn.Linear(6*2*2, 8), nn.Tanh(),
    nn.Linear(8, 4),
)
```

note_on_pytorch() is **print()** calls containing plain text, not executable code -- PyTorch is never imported anywhere in this file. The second **nn.Conv2d**'s **in_channels=3** matches the FIRST **Conv2d**'s **out_channels=3** exactly, mirroring **TwoBlockConvNet**'s **Wc2** shape from Segment 8.

```
def main():
    demo_multichannel_conv()
    demo_receptive_field()
    X, y = demo_dataset()
    demo_training(X, y)
    note_on_pytorch()

if __name__ == "__main__":
    main()
```

The same overall structure as every prior day: one **main()** calling each demo in the exact order it should run, with **X, y** from **demo_dataset()** explicitly threaded into **demo_training(X, y)** so both networks train on identical data, guarded by the standard **if __name__ == "__main__":** check.

Quick Syntax Glossary — New or Reinforced Constructs

`X[:, None, :, :]` (*axis insertion*) — Inserts a new length-1 axis at position 1, turning a plain (m,H,W) grayscale batch into the (m,1,H,W) shape a multi-channel conv function requires -- lets a single-channel image reuse the same `conv2d_forward_mc` built for arbitrary channel counts.

`dZ_ij[:,None,None,None]` (*4D broadcasting*) — Reshapes a (m,) array into (m,1,1,1) so it broadcasts against a (m,C_in,kH,kW)-shaped patch -- one more None than Day 41's 3D version, matching the new channel axis.

`array.size` (*total element count*) — Returns the product of every dimension in a NumPy array -- summed across every weight/bias tensor, this gives a network's total trainable parameter count directly, used for the parameter-efficiency comparison.

`for k, s in layers:` (*tuple unpacking in a loop*) — `layers` is a list of (kernel_size, stride) tuples; unpacking them directly in the loop header lets `receptive_field_growth` apply the same two-line update to any sequence of layers.

`rng.choice(4, size=n_arms, replace=False)` — Picks `n_arms` DIFFERENT values out of 4 without repeats -- used to choose which distinct directions a junction's arms point in, so a 2-arm image never draws the same direction twice.

`window.mean(axis=(2, 3))` (*average pooling*) — Spreads sensitivity across every pixel in a window equally, unlike max pooling's single-winner selection -- used only to verify the receptive-field formula, never for actual training.

`conv2d_backward_mc(...)[2]` / `dpooled1 capture` — The third return value (dX), always discarded as `_` in a single-block network, is captured here and reused directly as the previous block's incoming gradient -- the one line that makes backward-pass stacking possible.

`100 * (1 - p2 / p1)` (*percentage reduction*) — The standard formula for 'how much smaller is p2 than p1, as a percentage' -- computed from two real, measured parameter counts rather than hardcoded.

`f-string labels embedding computed values` — `ax.plot(..., label=f"OneBlockConvNet ({p1} params)")` embeds a variable's real value directly into a plot legend at draw time, so the chart always reflects whatever the code actually computed.

End of Day 42 syntax reference. For the stacking, receptive-field, and parameter-efficiency concepts these lines implement, see the main Day 42 study guide PDF.